

1. What is Arduino?

Definition

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It is designed to build electronic projects, automate tasks, and develop IoT applications. It contains a microcontroller that can read input from sensors, process the data, and control output devices such as LEDs, motors, and actuators.

History of Arduino

- Developed at the Interaction Design Institute Ivrea, Italy.
- Created as an easy tool for rapid prototyping.
- Designed especially for students and beginners who have little or no knowledge of electronics and programming.

Why is Arduino Popular?

Reason	Description
Open-source	Hardware and software are freely available to use and modify
Easy to use	Simple IDE; beginner-friendly programming based on C/C++
Low cost	Affordable boards available for all budgets
Large community	Huge online community with tutorials, libraries, and forums
Cross-platform	Works on Windows, macOS, and Linux
Extensible	Many shields and modules available for expansion

Applications of Arduino

Arduino is used in many real-world applications, such as: Home automation, Robotics, IoT (Internet of Things), Smart agriculture, Weather monitoring, Smart parking systems, Industrial automation, Embedded systems

Types of Arduino Boards

Category	Board Examples	Use Case
Entry-level	Arduino UNO, Arduino Nano	Learning, prototyping, simple projects
Enhanced	Arduino Mega, Arduino Leonardo	More pins, USB HID, complex projects
IoT / Wireless	Arduino MKR WiFi, Arduino Nano 33 IoT	IoT applications with built-in connectivity
Industrial / Pro	Arduino Due, Arduino Portenta H7	High performance, industrial automation

2. Arduino UNO

Definition

Arduino UNO is the most popular single-board microcontroller based on the ATmega328P microcontroller. It is suitable for learning embedded systems, electronics, and IoT.

Meaning of ATmega328P

Part	Meaning
AT	Atmel (manufacturer)
mega	Mega AVR family

328	Flash memory size group (32 KB)
P	Pico Power — low power consumption variant

Specifications of Arduino UNO

Specification	Value
Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7–12V
Digital I/O Pins	14 (6 with PWM output)
Analog Input Pins	6 (A0–A5)
Flash Memory	32 KB (0.5 KB used by bootloader)
SRAM	2 KB
EEPROM	1 KB
Clock Speed	16 MHz
USB Interface	Type-B USB
Communication	UART, SPI, I²C

Competitors of Arduino UNO

Some popular alternatives are: STM32 ARM Cortex, MSP430, PIC Microcontroller (PIC MCU)

3. Main Components (Pins and Jacks) of Arduino UNO

Arduino UNO contains several important pins and connectors.

Pin Description

1. Power Pins

Pin	Description
VIN	Input voltage pin; used to power the Arduino from an external power source (7–12V)
5V	Regulated 5V output; powers the microcontroller and connected components
3.3V	Regulated 3.3V output; for sensors that require 3.3V
GND	Ground pins (0V reference)
RESET	Resets the microcontroller when connected to GND

2. ICSP (In-Circuit Serial Programming)

ICSP Pins: MOSI, MISO, SCK, RESET, VCC, GND

Uses of ICSP:

Program the Arduino externally | Upload firmware | Connect multiple Arduino boards

3. Analog Pins

Pins	Function	Example Use
------	----------	-------------

A0–A5	Read analog voltage (0–5V); converted to digital value 0–1023 by 10-bit ADC	Temperature sensor, Light sensor, Gas sensor
-------	---	--

4. Digital Pins

Pins	Function	Example Use
0–13	Can be configured as INPUT or OUTPUT; reads or writes HIGH/LOW signals	Turn LED ON/OFF, Read switch status

5. Serial Communication Pins

Pin	Function
Pin 0 (RX)	Receive serial data
Pin 1 (TX)	Transmit serial data

Used for communication between Arduino and a computer or another device.

6. External Interrupt Pins

Pin	Interrupt	Example Use
Pin 2	INT0	Door sensor, Emergency button, Motion detection
Pin 3	INT1	Door sensor, Emergency button, Motion detection

7. PWM Pins (Pulse Width Modulation)

PWM Pins	Uses
3, 5, 6, 9, 10, 11	Motor speed control LED brightness control Servo motor control

8. SPI (Serial Peripheral Interface)

Pin	SPI Function
Pin 10	SS (Slave Select)
Pin 11	MOSI (Master Out Slave In)
Pin 12	MISO (Master In Slave Out)
Pin 13	SCK (Serial Clock)

Uses: High-speed communication with SD Cards, LCD displays, Sensors

9. Built-in LED

Pin 13 has a built-in LED on the Arduino UNO board. It is used for testing and debugging programs.

10. I²C (Inter-Integrated Circuit)

Also called Two-Wire Interface (TWI).

Pin	I ² C Function
A4	SDA (Serial Data)
A5	SCL (Serial Clock)

Uses: Communication with RTC modules, OLED displays, EEPROM, Sensors

11. AREF (Analog Reference)

The AREF (Analog Reference) pin provides an external reference voltage for analog-to-digital

conversion. Normally, Arduino uses 5V as the reference voltage. If an external reference is connected to AREF, analog readings are calculated based on that voltage instead.

Example: If AREF = 4V, then the `analogRead()` range of 0–1023 corresponds to 0–4V instead of 0–5V.

4. Arduino in IoT

Arduino can be used to build IoT systems, but the standard Arduino UNO does not have built-in Internet connectivity. Therefore, it requires additional hardware.

Network Option	Module / Shield	Description
WiFi	ESP8266, ESP32, WiFi Shield	Connects Arduino to WiFi networks for Internet access
Ethernet	Ethernet Shield (W5100)	Wired Internet connection via RJ45 cable
Bluetooth	HC-05, HC-06, BLE Module	Short-range wireless communication
GSM / GPRS	SIM800L, SIM900	Mobile network connectivity for remote IoT
LoRa	LoRa Shield, RFM95	Long-range, low-power IoT communication

5. Arduino IDE (Integrated Development Environment)

Definition

The Arduino IDE is the official software used to: Write Arduino programs | Compile the code | Upload the code to the Arduino board. Programs written in Arduino IDE are called Sketches.

Built-in Examples

Arduino IDE provides many example programs. Purpose: Learn Arduino programming | Understand sensors and actuators | Test hardware. How to Open: File → Examples

Setting the Arduino Board

Before uploading a program, select the correct board.

Steps:

- Connect Arduino using a USB cable.
- Open Arduino IDE.
- Select Tools → Board.
- Choose Arduino UNO (or your board model).

Setting the Serial Port

Arduino IDE must know which COM port the board is connected to.

Steps: Go to Tools → Port → Select the appropriate COM Port.

Tip: If unsure, unplug the Arduino and note which COM port disappears. That port belongs to the Arduino.

Code Compilation

Compilation is the process of converting the Arduino source code into machine code that the ATmega328P microcontroller can execute. If there are no errors, the IDE displays: "Compilation Successful"

Code Uploading

Uploading transfers the compiled program from the computer to the Arduino board.

Steps:

- Connect the Arduino board.
- Click the Upload button.

- Wait until the IDE displays "Done Uploading".
- The uploaded program starts running immediately.

Example: If the Blink sketch is uploaded successfully, the built-in LED connected to Pin 13 starts blinking.

Quick Exam Summary

Topic	Key Points
Arduino	Open-source hardware + software platform
Arduino UNO	ATmega328P 14 digital pins 6 analog pins 32KB Flash 16MHz
IDE	Write → Compile → Upload
Sketch	Arduino program
PWM Pins	3, 5, 6, 9, 10, 11
I ² C	A4 (SDA), A5 (SCL)
SPI	10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK)
Serial	0 (RX), 1 (TX)

Easy Memory Tricks

Arduino = Hardware + Software

IDE = Write → Compile → Upload

Sketch = Arduino Program

PWM Pins = 3, 5, 6, 9, 10, 11

I²C = A4 (SDA), A5 (SCL)

SPI = 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK)

Serial = 0 (RX), 1 (TX)

6. Arduino IDE — Serial Monitor & LED Blink

Serial Monitor

The Serial Monitor is a built-in tool in the Arduino IDE that acts as a communication interface between the computer and the Arduino board. It allows users to send text data to the Arduino and receive text data from the Arduino through serial communication. It is mainly used for debugging, displaying sensor values, and testing programs.

Features of Serial Monitor

Feature	Description
Two-way communication	Can both send data to and receive data from Arduino
Baud rate selection	Must match Serial.begin() baud rate in the sketch
Timestamp option	Can display timestamps with each message
Line ending options	None, Newline, Carriage Return, or Both
Text input field	Allows sending commands or data to the Arduino

How to Open Serial Monitor

- Connect the Arduino board to the computer.
- Open Arduino IDE.
- Select the correct Port: Tools → Port → COMxx

- Click the Serial Monitor icon at the top-right corner of the IDE.

Example

Suppose a temperature sensor is connected to Arduino. The Arduino reads the temperature and prints:

```
Temperature = 30°C
Humidity = 65%
```

These values appear in the Serial Monitor, allowing the user to observe sensor data in real time.

Serial Plotter

The Serial Plotter is another built-in tool in the Arduino IDE that displays serial output as a graph instead of plain text. It is useful for monitoring values that change continuously over time.

Features of Serial Plotter

Feature	Description
Graph output	Displays values as a real-time line graph instead of text
Multiple variables	Can plot multiple variables simultaneously in different colours
Auto-scaling	Automatically adjusts the Y-axis scale
Continuous update	Updates the graph in real time as new values arrive

Applications: Temperature monitoring, Humidity monitoring, Light intensity measurement, Heartbeat monitoring, Accelerometer data visualization

Example

A temperature sensor sends values: 25, 26, 27, 28, 29, 30. Instead of displaying these numbers as text, the Serial Plotter draws a graph showing how the temperature changes over time.

Installing Sensor Libraries

A library is a collection of pre-written functions and code that makes it easier to use sensors, modules, and other hardware without writing everything from scratch.

Example: To use the DHT11 Temperature and Humidity Sensor, we need the DHT.h library.

Method	Steps
Method 1: Library Manager (Recommended)	Open Arduino IDE → Sketch → Include Library → Manage Libraries → Search for DHT → Install DHT Sensor Library by Adafruit → Then install Adafruit Unified Sensor Library
Method 2: Import ZIP Library	Download the library in .zip format → Open Arduino IDE → Sketch → Include Library → Add .ZIP Library → Choose the downloaded ZIP file
Method 3: Manual Installation	Download the library → Extract the ZIP file → Copy the extracted folder into Sketchbook → Libraries folder → Restart Arduino IDE

LED Blink Program

The LED Blink Program is the first and simplest Arduino program. It turns an LED ON and OFF repeatedly after a fixed time interval. It is mainly used to verify that the Arduino board and IDE are working correctly.

Type	Description
Blink Built-in LED Continuously	LED remains continuously ON (BareMinimum example)

Blink Built-in LED Periodically	LED turns ON and OFF repeatedly after fixed delay (Blink example)
Blink External LED	External LED connected to Digital Pin 9 blinks periodically

Blink Built-in LED Continuously

Open Arduino IDE → File → Examples → Basics → BareMinimum → Upload the program.

Output: The built-in LED remains continuously ON.

Blink Built-in LED Periodically

Open Arduino IDE → File → Examples → Basics → Blink → Upload the program.

Output: The built-in LED turns ON and OFF repeatedly after a fixed delay.

Working Principle

The Arduino program performs the following steps repeatedly:

- Turn the LED ON.
- Wait for a specified time (e.g., 1 second).
- Turn the LED OFF.
- Wait again.
- Repeat forever.

Blink External LED — Circuit Connection

Component	Connection
LED (Anode +)	Connect to Digital Pin 9 through a 1 kΩ resistor
LED (Cathode −)	Connect to GND
1 kΩ Resistor	Between Digital Pin 9 and LED Anode

Why is a 1 kΩ Resistor Used? The resistor limits the current flowing through the LED. Without the resistor, excessive current may damage the LED or the Arduino pin.

Output: After uploading the Blink sketch, the external LED connected to Digital Pin 9 blinks periodically.

Arduino IDE Workflow

The complete process of running an Arduino program is:

- Write the sketch.
- Compile the sketch.
- Connect the Arduino board.
- Select the correct board.
- Select the correct COM port.
- Upload the sketch.
- Observe the output using: LED, Serial Monitor, or Serial Plotter

7. ATmega328P vs STM32 vs MSP430

Feature	ATmega328P (Arduino UNO)	STM32 (ARM Cortex-M)	MSP430
Manufacturer	Microchip (Atmel)	STMicroelectronics	Texas Instruments
Architecture	8-bit AVR	32-bit ARM Cortex-M	16-bit RISC

Clock Speed	16 MHz	Up to 216 MHz	Up to 25 MHz
Flash Memory	32 KB	Up to 2 MB	Up to 256 KB
SRAM	2 KB	Up to 512 KB	Up to 66 KB
Power Consumption	Medium	Medium–High	Ultra-low
Cost	Low	Medium	Low–Medium
Ease of Use	Very Easy (Arduino IDE)	Moderate (STM32CubeIDE)	Moderate (Code Composer)
Best For	Learning, prototyping	High-performance IoT	Battery-powered IoT

Advantages and Disadvantages

Board	Advantages	Disadvantages
ATmega328P (UNO)	Easy to use Large community Cheap Simple IDE	8-bit only Limited memory No built-in WiFi
STM32	32-bit power Fast Large memory Many peripherals	Complex setup Steeper learning curve
MSP430	Ultra-low power Good for battery-powered devices Low cost	Less community support Limited peripherals

Applications

Board	Typical Applications
ATmega328P (UNO)	LED blink, sensor reading, home automation, beginner robotics
STM32	Industrial IoT, motor control, real-time systems, high-speed data processing
MSP430	Wearable devices, energy harvesting, remote sensors, smartmeters

Easy Memory Tricks

Serial Monitor = Text Output

Serial Plotter = Graph Output

Library = Ready-made Code

Blink = First Arduino Program

Pin 13 = Built-in LED

Pin 9 = External LED Example

ATmega328P = Easy & Cheap

STM32 = Powerful & Fast

MSP430 = Low Power & Energy Efficient

8. Introduction to Arduino Programming

What is Arduino Programming?

Arduino Programming is the process of writing programs (called sketches) in the Arduino IDE to control the Arduino board and connected electronic devices such as LEDs, sensors, motors, and displays. Arduino programming is based on the C/C++ programming language, with additional built-in functions and libraries that simplify hardware control.

Arduino Sketch

An Arduino Sketch is a program written in the Arduino IDE and uploaded to the Arduino board. Every Arduino sketch must contain two mandatory functions: `setup()` and `loop()`

Structure of an Arduino Program

```
void setup()
{
    // Initialization code
}

void loop()
{
    // Main program
}
```

setup() Function

The `setup()` function is executed only once when the Arduino board is powered on or reset. It is mainly used for initialization.

Uses of `setup()`: Initialize serial communication | Set pin modes (INPUT or OUTPUT) | Initialize variables | Configure sensors and devices

Example

```
void setup()
{
    pinMode(13, OUTPUT);
    Serial.begin(9600);
}
```

Here, Pin 13 is configured as an output. Serial communication starts at 9600 baud.

loop() Function

The `loop()` function is executed continuously after `setup()` finishes. It contains the main logic of the program.

Uses of `loop()`: Read sensor values | Control LEDs and motors | Process data | Send and receive serial data | Perform repeated tasks

Example

```
void loop()
{
    digitalWrite(13, HIGH);
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}
```

The LED continuously turns ON and OFF every second.

Arduino Programming Language

Arduino programming is based on C/C++, so it supports almost all common programming features.

Feature	Description
Variables & Data Types	int, float, char, boolean, String, long, double
Operators	Arithmetic, Relational, Logical, Bitwise, Assignment
Control Structures	if, if-else, for, while, do-while, switch-case
Functions	User-defined functions; built-in Arduino functions
Arrays	Store multiple values of the same data type

Pointers	Memory address manipulation (from C/C++)
Libraries	Include external pre-written code using #include

Arduino Libraries

An Arduino Library is a collection of pre-written functions that make it easy to interface with sensors, displays, communication modules, motors, and other hardware. Libraries save programming time and reduce code complexity.

Examples: DHT Library, Servo Library, LiquidCrystal Library, WiFi Library

Variables

A variable is a named memory location used to store data that can change during program execution.

Example: `int temperature = 25;`

Common Data Types

Data Type	Size	Range / Description
int	2 bytes	-32,768 to 32,767
long	4 bytes	-2,147,483,648 to 2,147,483,647
float	4 bytes	Decimal numbers (6–7 significant digits)
double	4 bytes	Decimal numbers (same as float on Arduino UNO)
char	1 byte	Single character (e.g., 'A')
boolean	1 byte	true or false
byte	1 byte	0 to 255
String	Variable	Sequence of characters (text)

Operators

Operator Type	Operators	Example
Arithmetic	+, −, *, /, %	a + b, a * b, a % b
Relational	==, !=, >, <, >=, <=	a == b, a > b
Logical	&&, , !	a && b, !a
Bitwise	&, , ^, ~, <<, >>	a & b, a << 2
Assignment	=, +=, -=, *=, /=	a = 5, a += 3

9. Built-in Arduino Functions

1. pinMode()

Syntax: `pinMode(pin, mode);`

Configures a digital pin as INPUT, OUTPUT, or INPUT_PULLUP. By default, all digital pins are configured as INPUT.

Mode	Description
OUTPUT	Pin drives HIGH (5V) or LOW (0V)
INPUT	Pin reads external signal; high impedance
INPUT_PULLUP	Pin reads external signal; internal pull-up resistor enabled (pin is HIGH by default)

Example: `pinMode(9, OUTPUT);` — Pin 9 is configured as an output.

2. `digitalRead()`

Syntax: `digitalRead(pin);`

Reads the current logic level from a digital input pin.

Return Value	Meaning
HIGH (1)	Pin voltage is approximately 5V
LOW (0)	Pin voltage is approximately 0V

Example: `int value = digitalRead(2);` — Reads the state of digital pin 2.

3. `digitalWrite()`

Syntax: `digitalWrite(pin, value);`

Writes a digital output to a pin.

Value	Output
HIGH	Sets pin to 5V
LOW	Sets pin to 0V

Example: `digitalWrite(13, HIGH);` — Turns ON the LED connected to pin 13.

4. `analogRead()`

Syntax: `analogRead(pin);`

Reads an analog voltage from analog pins (A0–A5). Arduino UNO has a 10-bit ADC (Analog-to-Digital Converter). The reading ranges from 0 to 1023.

Resolution: Arduino UNO converts 0–5V → 0–1023. Each step represents approximately $5V \div 1024 \approx 4.88 \text{ mV}$.

Example: `int value = analogRead(A3);` — Reads the analog voltage from pin A3.

`analogReference()`

Changes the reference voltage used by `analogRead()`. Normally, Arduino uses 5V as the reference voltage. An external reference can also be used.

5. `analogWrite()`

Syntax: `analogWrite(pin, value);`

Generates a PWM (Pulse Width Modulation) signal to simulate analog output. PWM value ranges from 0 to 255.

Note: `analogRead()` returns 0–1023. `analogWrite()` accepts 0–255. Therefore, sensor values are often divided by 4 before writing.

Example: `analogWrite(9, value / 4);`

6. `delay()`

Syntax: `delay(millisecods);`

Pauses program execution for the specified time.

Example: `delay(1000);` — Waits for 1 second.

10. Serial Communication Functions

Serial communication allows data exchange between the Arduino and a computer.

Function	Syntax	Description
<code>Serial.begin()</code>	<code>Serial.begin(9600);</code>	Starts serial communication at the specified baud rate

Serial.available()	Serial.available();	Returns the number of bytes available to read from the serial buffer
Serial.print()	Serial.print(value);	Displays data on the Serial Monitor in a human-readable format
Serial.println()	Serial.println(value);	Works like Serial.print(), but moves the cursor to the next line after printing
Serial.read()	char c = Serial.read();	Reads one byte of incoming serial data
Serial.write()	Serial.write(45);	Sends raw binary data (bytes) through the serial port; sends actual byte values, not ASCII text

Serial.print() with Format

Syntax: Serial.print(value, format);

Format	Description
BIN	Prints in binary
OCT	Prints in octal
DEC	Prints in decimal (default)
HEX	Prints in hexadecimal

Example: Serial.print(number, HEX); — Prints the number in hexadecimal.

11. Mathematical & Trigonometric Functions

Function	Description	Example
abs(x)	Absolute value of x	abs(-5) = 5
constrain(x, a, b)	Constrains x to the range [a, b]	constrain(120, 0, 100) = 100
map(x, in_min, in_max, out_min, out_max)	Maps x from one range to another	map(512, 0, 1023, 0, 255) = 127
max(a, b)	Returns the larger of a and b	max(3, 7) = 7
min(a, b)	Returns the smaller of a and b	min(3, 7) = 3
pow(base, exp)	Returns base raised to the power exp	pow(2, 3) = 8
sq(x)	Returns the square of x	sq(4) = 16
sqrt(x)	Returns the square root of x	sqrt(16) = 4

Trigonometric Function	Description
sin(x)	Returns the sine of x (x in radians)
cos(x)	Returns the cosine of x (x in radians)
tan(x)	Returns the tangent of x (x in radians)

12. Example: Digital Read–Write

Objective

Turn an LED ON or OFF using a push button.

Circuit Connections

Component	Pin	Connection
Push Button (one side)	Pin 2	Connected to 5V when pressed
10 kΩ Pull-down Resistor	Pin 2 to GND	Keeps Pin 2 LOW when button is not pressed
LED (Anode)	Pin 13	Through 220Ω resistor
LED (Cathode)	GND	Connect to GND

Working Principle

When the Button is Not Pressed

The switch is open. Pin 2 is connected to Ground through the 10 kΩ pull-down resistor. Arduino reads LOW (0V). LED remains OFF.

When the Button is Pressed

The switch closes. Pin 2 connects directly to 5V. Arduino reads HIGH. LED turns ON.

Why is a Pull-down Resistor Needed?

Without a pull-down resistor, the input pin may float, meaning its voltage is undefined and it can randomly read HIGH or LOW due to electrical noise. The pull-down resistor keeps the pin at LOW when the button is not pressed, ensuring stable and reliable readings.

Quick Exam Summary

Topic	Key Points
Sketch	Arduino Program = setup() + loop()
setup()	Runs Once — initialization
loop()	Runs Forever — main logic
digitalRead()	Read Digital Input; returns HIGH or LOW
digitalWrite()	Control LED or Device; write HIGH or LOW
analogRead()	Returns 0–1023 (10-bit ADC)
analogWrite()	PWM output 0–255
delay(1000)	Wait 1 Second
Serial.print()	Print on Same Line
Serial.println()	Print then move to Next Line
Button Pressed	Reads HIGH
Button Released	Reads LOW (with Pull-down Resistor)

Easy Memory Tricks

Sketch = Arduino Program

setup() = Runs Once

loop() = Runs Forever

digitalRead() = Read Digital Input

digitalWrite() = Control LED or Device

analogRead() = 0–1023

analogWrite() = PWM (0–255)
delay(1000) = Wait 1 Second
Serial.print() = Same Line
Serial.println() = Next Line
Button Pressed = HIGH
Button Released = LOW (with Pull-down Resistor)

13. Example 2: Binary Counter using LEDs

A Binary Counter is a circuit that displays binary numbers (0s and 1s) using LEDs. Each LED represents one binary bit. As the counter increases, the LEDs turn ON and OFF according to the binary value.

Requirements

Component	Quantity
Arduino UNO	1
LEDs	4
1 kΩ Resistors	4
Breadboard	1
Jumper Wires	As required
USB Cable	1

Circuit Connections

- Place the LEDs and resistors on the breadboard.
- Connect the breadboard power rails to the Arduino's 5V and GND.
- Connect each LED to a digital output pin of the Arduino through a 1 kΩ resistor.
- Connect the Arduino to the computer using a USB cable.

Programming Steps

- Install the Arduino IDE.
- Open the Arduino IDE.
- Select Arduino UNO from Tools → Board.
- Select the correct COM Port.
- Write the sketch (program).
- Compile and upload the program.
- The LEDs will blink in binary counting order.

Working Principle

The Arduino sends HIGH and LOW signals to four LEDs. Each LED represents one binary bit.

Decimal	Binary (D3 D2 D1 D0)	LED State
0	0000	All OFF
1	0001	D0 ON
2	0010	D1 ON
3	0011	D1, D0 ON
4	0100	D2 ON
...	...	...
15	1111	All ON

The LEDs continuously count from 0000 to 1111, then repeat.

14. Reading Analog Voltage

Many sensors produce analog voltages, but Arduino can only process digital values. Therefore, Arduino uses an Analog-to-Digital Converter (ADC) to convert analog voltage into a digital number.

ADC (Analog-to-Digital Converter)

Arduino UNO has a 10-bit ADC. This means: Input Voltage = 0–5 V | Digital Output = 0–1023

Parameter	Value
ADC Resolution	10-bit
Input Voltage Range	0–5V
Digital Output Range	0–1023
Voltage per Step	$5V \div 1024 \approx 4.88 \text{ mV}$

Formula for Converting Digital Value to Analog Voltage

Analog Voltage = (Digital Value × Vref) / (2ⁿ – 1)

Where: Digital Value = value returned by analogRead() | Vref = Reference Voltage (5V for Arduino UNO) | n = ADC resolution in bits (10 for Arduino UNO)

For Arduino UNO: Analog Voltage = (Digital Value × 5) / 1023

Example

Given: Digital Value = 512, Reference Voltage = 5V

Calculation: Analog Voltage = $(512 \times 5) / 1023 = 2.5V$

Answer: The input analog voltage is approximately 2.5 V.

analogRead()

Syntax: digitalValue = analogRead(pin);

Reads the analog voltage from an analog pin and converts it into a digital value.

Parameter	Description
pin	Analog pin number (A0–A5)
Return Value	Integer between 0 and 1023

Example: Reading Analog Voltage using a Potentiometer

Component	Connection
Potentiometer (left pin)	Connect to GND
Potentiometer (middle pin / wiper)	Connect to Arduino A0
Potentiometer (right pin)	Connect to 5V

Program Working

- Read the analog value from pin A0 using analogRead().
- Store the value in the variable digitalValue.
- Convert the digital value into analog voltage using the formula: $\text{analogVoltage} = (\text{digitalValue} \times 5.00) / 1023.00$;
- Print both the digital value and analog voltage on the Serial Monitor.
- Wait for 1 second using delay(1000) and repeat.

Sample Output

```
digital value = 350    analog voltage = 1.71 V  
digital value = 800    analog voltage = 3.91 V
```

As the potentiometer knob is rotated, the analog voltage changes, and Arduino displays the corresponding digital value and voltage.

15. Writing Analog Value (PWM)

Arduino cannot generate a true analog voltage. Instead, it uses Pulse Width Modulation (PWM) to simulate analog output. PWM rapidly switches the output between HIGH (5V) and LOW (0V). By changing the percentage of ON time (called the duty cycle), Arduino creates the effect of different analog voltages.

analogWrite()

Syntax: `analogWrite(pin, value);`

Generates a PWM signal on PWM-supported digital pins.

Value	Duty Cycle	Equivalent Voltage
0	0%	0V (always LOW)
127	50%	~2.5V
255	100%	5V (always HIGH)

PWM Pins in Arduino UNO: 3, 5, 6, 9, 10, and 11

Example: LED Brightness Control

Component	Connection
Potentiometer (wiper)	Connect to A0
LED (Anode)	Connect to Pin 9 through 220Ω resistor
LED (Cathode)	Connect to GND

Working Principle

- The potentiometer changes the analog voltage.
- Arduino reads the voltage using `analogRead()`.
- Since `analogRead()` returns 0–1023, divide the value by 4 to convert it to the PWM range 0–255.
- Send the PWM value to the LED using `analogWrite()`.
- The LED brightness changes according to the potentiometer position.

Example Code

```
int value = analogRead(A0);  
analogWrite(9, value / 4);
```

Output

Turning the potentiometer changes the analog voltage. Arduino converts it into a PWM signal. The LED gradually becomes brighter or dimmer.

Difference Between `analogRead()` and `analogWrite()`

Feature	<code>analogRead()</code>	<code>analogWrite()</code>
Purpose	Reads analog input voltage	Outputs PWM signal (simulated analog)
Pins	A0–A5 (analog input pins)	3, 5, 6, 9, 10, 11 (PWM pins)

Value Range	0 to 1023	0 to 255
Resolution	10-bit ADC	8-bit PWM
Direction	Input (reads from sensor)	Output (controls device)
Conversion needed	Divide by 4 to use with analogWrite()	Receives divided value from analogRead()

16. Lessons Learned — Exam Summary

Topic	Key Points
Binary Counter	Uses LEDs to display binary numbers from 0000 to 1111
ADC	Converts analog voltage into a digital value
Arduino UNO ADC	10-bit resolution, producing values from 0 to 1023
Analog Voltage Formula	$\text{Voltage} = (\text{Digital Value} \times 5) / 1023$
analogRead()	Reads analog input from pins A0–A5; returns 0–1023
PWM	Simulates analog output by varying the duty cycle of a digital signal
analogWrite()	Outputs PWM values from 0 to 255 on PWM pins (3,5,6,9,10,11)
LED Dimming	A potentiometer controls LED brightness by reading an analog value and writing a corresponding PWM value